

Práctica 4: Herramientas de análisis estático de código

Análisis del proyecto *ajedrez* con PMD

Iván Rodríguez Miranda

26 de mayo de 2026

Herramienta	PMD 7.7.0 (Maven PMD Plugin 3.26.0)
Proyecto evaluado	ajedrez (Java 21, Maven)
Fecha del análisis	09/05/2026
Archivos analizados	21 ficheros .java
Líneas de código	1145 líneas en src/main/java

Índice

1. Objetivo de la práctica	3
2. Descripción del código Java evaluado	3
2.1. Lógica de negocio e interfaz de usuario implementadas	4
3. Instalación y configuración de PMD	5
4. Reglas utilizadas y justificación	5
5. Secuencia de comandos ejecutados	6
6. Resultados obtenidos con PMD y Referencia a los Reportes del Programa	7
6.1. El Reporte Estructurado XML: target/pmd-original.xml	7

6.2. El Reporte Interactivo HTML: target/reports/pmd-original.html	8
6.3. Resumen de las violaciones detectadas	8
7. Listado e interpretación de los errores detectados	8
7.1. Ficha de Violación: NullAssignment (Error Prone)	9
7.2. Ficha de Violación: CompareObjectsWithEquals (Error Prone)	9
7.3. Ficha de Violación: LocalVariableCouldBeFinal (Code Style)	10
7.4. Ficha de Violación: UnnecessaryConstructor (Code Style)	10
7.5. Ficha de Violación: AvoidInstantiatingObjectsInLoops (Performance)	11
8. Relación con la calidad del software según ISO/IEC 25000	11
9. Conclusiones y resolución de hallazgos	12
Anexo A: Evidencias de salida	14
Anexo B: Materiales y bibliografía	14

1. Objetivo de la práctica

El objetivo fundamental de esta práctica consiste en el despliegue, configuración y aplicación de una **herramienta de análisis estático de código** sobre un sistema de software desarrollado en lenguaje Java. En este caso particular, se ha seleccionado la herramienta de código abierto **PMD** (versión 7.7.0) integrada dentro del flujo de compilación mediante el plugin de Maven. Los resultados arrojados por el análisis se interpretan formalmente para relacionar las advertencias y fallos detectados con los criterios de calidad interna definidos por el estándar internacional **ISO/IEC 25000** (Quality Improvement Paradigm y el modelo de calidad de producto SQuaRE).

A diferencia de las técnicas de análisis dinámico, que dependen de la ejecución del programa mediante casos de prueba (testing), el análisis estático inspecciona la estructura lógica de los ficheros de código fuente directamente. PMD realiza esta tarea analizando el código y transformándolo en un **Árbol de Sintaxis Abstracta** (AST - *Abstract Syntax Tree*). Sobre los nodos de este árbol, el motor de la herramienta evalúa reglas semánticas y sintácticas escritas en XPath o clases Java, identificando de manera automatizada:

- Defectos y patrones de código propensos a fallos en tiempo de ejecución (**Error Prone**).
- Variables, campos o métodos declarados que no se utilizan en ningún flujo (**Best Practices**).
- Construcciones sintácticas ineficientes que degradan el rendimiento (**Performance**).
- Estilos de codificación confusos o contrarios a las convenciones estándar de Java (**Code Style**).

La justificación de la Calidad Interna

La calidad externa de un sistema (lo que percibe el usuario al interactuar con el ajedrez) depende directamente de su calidad interna (la legibilidad, inmutabilidad y modularidad del código fuente). El estándar **ISO/IEC 25010** establece que la **Mantenibilidad** es la característica clave para medir el esfuerzo necesario para corregir, adaptar o hacer evolucionar un software en el futuro.

2. Descripción del código Java evaluado

El sistema de software bajo análisis es una implementación interactiva y funcional del clásico juego de ajedrez desarrollado en el lenguaje de programación **Java** (específicamente bajo la especificación sintáctica de JDK 21) y haciendo uso de la biblioteca multimedia y de interfaz de usuario **JavaFX**. La arquitectura interna del programa sigue el patrón de diseño clásico de desacoplamiento **Modelo-Vista-Controlador (MVC)**.

El **Modelo** se agrupa bajo el paquete lógico `com. ajedrez. modelo` y sus clases de datos. Representa la lógica interna del dominio del ajedrez, conteniendo la matriz bidimensional de casillas, la distribución del tablero, los turnos de juego, y las restricciones de desplazamiento individuales para cada una de las seis piezas clásicas (Alfil, Caballo, Peón, Reina, Rey y Torre). Toda esta lógica se mantiene aislada de la visualización física.

El **Controlador** se ubica bajo el paquete `com. ajedrez. controlador`. Actúa como mediador central recibiendo los eventos físicos y de pulsación de la vista. Comprueba en el modelo la validez del movimiento solicitado, actualiza el estado de las casillas (incluyendo la lógica de jaques, jaque mates y tablas) y devuelve las respuestas visuales.

La **Vista** se agrupa en el paquete `com. ajedrez. vista` y utiliza los componentes visuales de JavaFX para dibujar en pantalla el tablero de ajedrez de 8x8 casillas con sus respectivas piezas blancas y negras, gestionando de forma gráfica la selección de las casillas y los estados interactivos mediante eventos de ratón.

El proyecto consta de 21 archivos fuente con extensión `. java` que representan un total aproximado de 1145 líneas de código ubicadas en la carpeta `src/main/ java`. Las métricas del entorno de desarrollo durante la fase de análisis corresponden al uso del JDK 21 para la compilación de las clases y del entorno de ejecución OpenJDK 25.0.2 bajo el sistema de construcción automatizada de Maven 3.8.5.

2.1. Lógica de negocio e interfaz de usuario implementadas

Antes de proceder a la ejecución de las herramientas de auditoría estática, se realizaron tareas de desarrollo para dar robustez a la lógica de negocio y la presentación de la aplicación:

- **Torre.java y Reina.java:** Se implementó su lógica de movimiento como subclases de la clase abstracta `Pieza`. La torre valida el desplazamiento en línea recta y la reina combina las diagonales y rectas, controlando en ambos casos el bloqueo por piezas intermedias.
- **Rey.java y Caballo.java:** El caballo implementa el salto característico en L y el rey su movimiento unitario, además de gestionar las reglas del enroque corto y largo de forma combinada con el estado de las torres.
- **Peon.java:** Controla el sentido vertical de avance según el color del bando (blanco o negro), el doble avance desde la fila de salida, el bloqueo frontal por piezas y la captura diagonal estándar.
- **VistaTablero.java y ControladorJuego.java:** Implementan la escena de JavaFX con estilos en el archivo CSS `ajedrez. css`. El controlador mantiene el estado de jaque de manera reactiva recalculando las amenazas sobre el rey tras cada movimiento.
- **package-info.java:** Se añadieron descripciones Javadoc detalladas a nivel de cada paquete para explicar la responsabilidad de cada capa y cumplir con los criterios de legibilidad de las APIs.

3. Instalación y configuración de PMD

La herramienta de análisis estático se ha incorporado directamente dentro del ciclo de vida de compilación y empaquetado del proyecto utilizando el plugin de Maven **maven-pmd-plugin** (versión 3.26.0). La integración en el archivo `pom.xml` se asocia a la fase de construcción de Maven mediante la etiqueta `<build>`, vinculando el goal de ejecución `check` a la fase estándar `verify`.

Adicionalmente, se configuró el plugin en la sección de generación de informes `<reporting>` de Maven. Esto permite que, al ejecutar el comando global de documentación `mvn site`, se invoque a PMD y se construya una suite de reportes interactivos en formato HTML.

Las opciones de configuración aplicadas dentro del plugin en el archivo XML de Maven definen los siguientes parámetros de comportamiento:

- El parámetro **targetJdk** se configuró en la versión 21 de Java. Esto fuerza al analizador sintáctico de PMD a validar la compatibilidad sintáctica de las nuevas características de Java (como los registros o el switch extendido) utilizadas en el código fuente.
- El parámetro **failOnViolation** se estableció en `false`. Esto permite que el ciclo de construcción de Maven complete la build con éxito (`BUILD SUCCESS`) aunque se detecten violaciones de estilo, permitiendo generar todos los reportes de calidad para su posterior análisis manual sin bloquear al desarrollador.
- Se configuró la exclusión explícita del punto de entrada `com/ajedrez/App.java` y de toda la jerarquía de interfaz de usuario de JavaFX bajo el paquete `com/ajedrez/vista/**`. Esta exclusión se justifica porque en el entorno de desarrollo la JVM de Maven es la versión OpenJDK 25.0.2, provocando que el motor de resolución de tipos de PMD arroje fallos internos debido a la incompatibilidad sintáctica temporal con la versión de bytecode de las librerías gráficas de JavaFX. Al excluirlas, el análisis se mantiene estable y enfocado sobre las clases de negocio (el modelo de piezas y los controladores de juego).
- Se definió el parámetro **rulesets** apuntando al archivo local de la raíz del proyecto denominado `pmd-ruleset.xml`, permitiendo aplicar un conjunto de reglas personalizado adaptado a la práctica.

4. Reglas utilizadas y justificación

La configuración de auditoría utiliza un ruleset personalizado definido en el archivo local **pmd-ruleset.xml** para el filtrado de las categorías oficiales de reglas de PMD. La justificación de las categorías seleccionadas bajo el estándar de calidad **ISO/IEC 25010** se describe a continuación:

La categoría **Best Practices** se utiliza para la identificación de malas prácticas de programación tradicionales en Java. Detecta el desuso de variables locales, scopes de visibilidad inadecuados o parámetros de métodos obsoletos, ayudando a la subcaracterística de **Analizabilidad** de la Mantenibilidad. Se excluyeron las reglas de aserciones de JUnit (`JUnitTestsShouldIncludeAssert`, etc.) para evitar falsos positivos en clases auxiliares.

La categoría **Code Style** unifica las convenciones de escritura del código. Se enfoca en forzar el uso de variables locales inmutables y la eliminación de código ruidoso. Para no sobrecargar la declaración de firmas en el estilo académico del código, se excluyó la regla `MethodArgumentCouldBeFinal`, permitiendo mantener los parámetros de los métodos sin el modificador **final**.

La categoría **Design** se ha configurado de manera estratégica. Se centra en detectar problemas de acoplamiento rígido, mala cohesión de objetos o abstracciones débiles. Con el fin de separar el análisis estructural de estilo de la medición cuantitativa de complejidad cíclica, se excluyeron explícitamente las reglas sintácticas de complejidad de métodos (`CognitiveComplexity`, `CyclomaticComplexity`, `NPathComplexity`, `TooManyMethods`) y el anidamiento profundo de condicionales (`AvoidDeeplyNestedIfStmts`, `CollapsibleIfStatements`).

La categoría **Error Prone** es la más crítica para la estabilidad del software, pues analiza patrones de código que tienen una alta probabilidad de provocar excepciones y fallos en ejecución. Se relaciona directamente con la **Fiabilidad** y la tolerancia a fallos del sistema. Se excluyó la regla `AvoidFieldNameMatchingMethodName` para evitar que el campo de inmutabilidad `haMovido` en las piezas de ajedrez chocase con su método getter equivalente `haMovido()`.

Las categorías **Multithreading** y **Performance** auditan la concurrencia y la eficiencia en la utilización de los recursos del sistema de memoria (montón o heap). Se asocian con la característica de **Eficiencia de desempeño** del producto software.

5. Secuencia de comandos ejecutados

El proceso de validación y generación de reportes se instrumentó ejecutando la siguiente secuencia de comandos desde la terminal en el directorio raíz del proyecto ajedrez:

En primer lugar, se ejecutó `java -version` para verificar el entorno local de la máquina virtual de Java, devolviendo la versión activa OpenJDK 25.0.2 (bytecode major version 69). Seguidamente, se ejecutó `./mvnw -version` para confirmar que el wrapper del proyecto cargase la versión compatible de Apache Maven 3.8.5.

A continuación, se invocó `./mvnw clean pmd:pmd` para limpiar la carpeta de compilación previa `target` y ejecutar el validador estático de PMD de forma directa sobre las fuentes Java no excluidas. Este paso compila las clases y genera el reporte estructurado inicial en formato XML en la ruta de compilación.

Para integrar el análisis dentro de las fases estándar de calidad del proyecto, se ejecutó `./mvnw verify`. Este comando ejecuta todas las fases de compilación y empaquetado, lanzando PMD de forma síncrona dentro del ciclo de verificación. Finalmente, se ejecutó `./mvnw site` para construir y renderizar el sitio web de documentación HTML navegable del proyecto ajedrez.

6. Resultados obtenidos con PMD y Referencia a los Reportes del Programa

Una vez completado el flujo de ejecución de la secuencia de comandos de Maven, el sistema de análisis estático produce los reportes oficiales del programa en la carpeta de compilación `target`. Estos reportes constituyen la **evidencia objetiva y la fuente de verdad primaria** para la evaluación de la calidad interna de la práctica. Con el fin de preservar el reporte inicial de 57 violaciones al corregir posteriormente el código fuente, se ha guardado una copia de respaldo de los reportes originales en las rutas **`target/pmd-original.xml`** y **`target/reports/pmd-original.html`**. Por su parte, los archivos estándar en las rutas por defecto corresponden al reporte final una vez limpio de violaciones el proyecto.

6.1. El Reporte Estructurado XML: `target/pmd-original.xml`

El reporte primario generado por el analizador es el archivo estructurado **`target/pmd-original.xml`** (inicialmente `target/pmd.xml`). Este archivo contiene el volcado de datos XML con todas las violaciones del código detectadas por PMD. Su estructura incluye el elemento raíz `<pmd>` con información de la versión y la marca de tiempo, seguido de una colección de elementos `<file>` indexados por la ruta absoluta de cada clase de Java analizada.

Dentro de cada archivo, se anidan elementos `<violation>` que documentan cada hallazgo mediante atributos estructurados:

- `beginline` y `endline`: Indican el rango de líneas exacto donde ocurre el problema en el código fuente.
- `rule` y `ruleset`: Indican el nombre de la regla infringida y la categoría general de PMD.
- `priority`: El nivel de severidad asignado por PMD (en este proyecto, prioridad 3 - severidad baja/media).
- El cuerpo de texto del nodo contiene el mensaje descriptivo para el programador.

Este archivo XML es fundamental porque permite la integración con herramientas externas de desarrollo continuo y cuadros de mando automáticos.

6.2. El Reporte Interactivo HTML: `target/reports/pmd-original.html`

Para facilitar la auditoría y corrección por parte de los desarrolladores, se procesa el XML anterior y se genera el reporte navegable interactivo `target/reports/pmd-original.html` (inicialmente `target/reports/pmd.html`).

Este archivo HTML contiene una interfaz web estructurada que:

- Muestra un resumen general con el número total de violaciones agrupadas por archivo de código fuente Java. * Permite navegar de forma jerárquica para desplegar las violaciones de cada clase. * Muestra la línea de código exacta afectada junto con el mensaje explicativo de la regla de PMD. * Incorpora hipervínculos directos a la documentación oficial de PMD para cada regla, lo que facilita al programador la comprensión del patrón erróneo y su resolución recomendada.

Ambos reportes confirman la existencia de un total de **57 violaciones** del ruleset aplicadas sobre el core lógico de la aplicación (excluyendo la interfaz gráfica JavaFX).

6.3. Resumen de las violaciones detectadas

El total de 57 violaciones arrojadas por los reportes oficiales del programa se distribuyen de la siguiente forma:

La categoría **Code Style** acumula la mayor parte de los avisos con **39 violaciones**. Esto se desglosa en 38 ocurrencias de la regla `LocalVariableCouldBeFinal` distribuidas de manera homogénea por todas las clases del modelo y del controlador, y 1 ocurrencia de la regla `UnnecessaryConstructor` en la clase `ControladorTablero.java`.

La categoría **Error Prone** acumula **17 violaciones**. Esto corresponde a 13 ocurrencias de la regla `CompareObjectsWithEquals` al comparar el enumerado de color de piezas con operadores de igualdad por referencia, y 4 ocurrencias de la regla `NullAssignment` al inicializar o liberar estados de casillas con el valor nulo.

La categoría **Performance** presenta únicamente **1 violación** de la regla `AvoidInstantiatingObjectsInLoops` en el método de inicialización de la clase `Tablero.java`.

7. Listado e interpretación de los errores detectados

A continuación se realiza el análisis e interpretación teórica detallada de cada una de las 5 violaciones que se disparan en el código, referenciando los hallazgos descritos en el reporte XML oficial del programa:

7.1. Ficha de Violación: NullAssignment (Error Prone)

De acuerdo con el reporte oficial original en `target/pmd-original.xml`, se detectan 4 violaciones de la regla `NullAssignment`. El caso más representativo ocurre en la clase `ControladorJuego.java`, línea 77, dentro del método `reiniciarPartida()`:

```
public void reiniciarPartida() {  
    tablero.inicializarPiezas();  
    turnoActual = ColorPieza.BLANCO;  
    casillaSeleccionada = null; // <- Asignacion a null  
}
```

La asignación explícita del valor `null` a una referencia de objeto indica la ausencia de un estado válido. En el diseño orientado a objetos moderno, esto se considera un síntoma de diseño mejorable. Fuerza a que todas las clases que interactúen con la variable tengan que realizar comprobaciones preventivas antes de llamar a sus métodos, aumentando de forma significativa el riesgo de que ocurra una excepción no controlada de tipo `NullPointerException` en tiempo de ejecución.

La solución recomendada en la ingeniería de software consiste en el uso de contenedores explícitos de opcionalidad, modificando el tipo de dato de la variable a `Optional<Casilla>`:

```
// Refactorizacion sugerida utilizando Optional  
private Optional<Casilla> casillaSeleccionada = Optional.empty();  
  
public void reiniciarPartida() {  
    tablero.inicializarPiezas();  
    turnoActual = ColorPieza.BLANCO;  
    casillaSeleccionada = Optional.empty(); // <- Limpio de nulos  
}
```

7.2. Ficha de Violación: CompareObjectsWithEquals (Error Prone)

El reporte XML oficial del programa identifica 13 violaciones de la regla `CompareObjectsWithEquals` distribuidas en las clases del modelo de piezas (por ejemplo, en la clase `Peon.java`, línea 83, al validar la captura):

```
return avanceFila == obtenerDireccion()  
    && avanceColumna == 1  
    && !destino.estaVacía()  
    && destino.getPiezaActual().getColor() != getColor(); // <- != en  
objetos
```

El operador `!=` o `==` en Java realiza una comparación de identidad por dirección de referencia física de memoria. Para verificar la equivalencia estructural de los datos internos de un objeto, se debe emplear

el método de equivalencia `.equals()`.

En esta implementación, dado que `ColorPieza` se modela como un enumerado (`enum`), la máquina virtual de Java garantiza la existencia de instancias únicas en memoria, haciendo que la comparación por referencia sea segura y eficiente. No obstante, PMD arroja el aviso para forzar al desarrollador a mantener la uniformidad estilística del lenguaje y prevenir errores graves en refactorizaciones de clases normales que dejen de ser enumerados.

7.3. Ficha de Violación: `LocalVariableCouldBeFinal` (Code Style)

Esta violación es la más numerosa en el reporte, acumulando 38 ocurrencias repartidas en las clases del modelo (por ejemplo, en la clase abstracta `Pieza.java`, líneas 52-53, al verificar si el camino está libre):

```
int avanceFila = Integer.compare(destino.getFila(), origen.getFila());
int avanceColumna = Integer.compare(destino.getColumna(),
    origen.getColumna());
```

Cualquier variable local que se inicialice una vez y cuyo valor no sea reasignado en el resto del cuerpo del método debería ser declarada con el modificador de inmutabilidad `final`. Esto hace explícito que la variable actúa como una constante local de solo lectura, mejorando la comprensión y evitando modificaciones accidentales durante tareas de mantenimiento.

Además, permite al compilador del JDK aplicar optimizaciones sintácticas en la pila de variables locales. La refactorización consiste en anteponer el modificador:

```
final int avanceFila = Integer.compare(destino.getFila(),
    origen.getFila());
final int avanceColumna = Integer.compare(destino.getColumna(),
    origen.getColumna());
```

7.4. Ficha de Violación: `UnnecessaryConstructor` (Code Style)

El reporte oficial de PMD indica 1 violación de la regla `UnnecessaryConstructor` en la clase `ControladorTablero.java`, línea 22:

```
public class ControladorTablero {
    public ControladorTablero() {
        // Constructor vacio redundante
    }
}
```

El compilador de Java autogenera un constructor público por defecto sin parámetros al compilar a bytecode si la clase no define explícitamente ninguno. Escribir un constructor vacío manualmente es

redundante y añade líneas de código innecesarias que ensucian el fichero. Eliminarlo limpia la estructura de la clase y mantiene el mismo comportamiento operativo.

7.5. Ficha de Violación: AvoidInstantiatingObjectsInLoops (Performance)

El reporte XML identifica 1 violación de la regla AvoidInstantiatingObjectsInLoops en la clase Tablero.java, línea 117, al inicializar la fila de peones:

```
private void inicializarPeones(final ColorPieza color, final int fila) {  
    for (int columna = 0; columna < TAMANO_TABLERO; columna++) {  
        colocarPieza(fila, columna, new Peon(color)); // <- new en bucle  
    }  
}
```

La instanciación sistemática de nuevos objetos utilizando el operador new dentro de bucles puede provocar problemas de rendimiento si el número de iteraciones es elevado, ya que acumula gran cantidad de objetos efímeros en el montón de memoria dinámica (*heap*) y satura el recolector de basura (*Garbage Collector*).

En esta fase de inicialización de la partida de ajedrez, dado que cada casilla del tablero debe representar una pieza física distinta con sus propios atributos y estado, la creación de instancias individuales es necesaria y no puede ser optimizada. Esta violación es tolerable y correcta en la carga inicial del juego.

8. Relación con la calidad del software según ISO/IEC 25000

La familia de estándares **ISO/IEC 25000** (SQuaRE) propone evaluar la calidad del producto software a partir de características internas y externas. Los hallazgos del análisis estático de PMD se relacionan de forma directa con los siguientes atributos del modelo **ISO/IEC 25010**:

La característica de **Mantenibilidad** se asocia de forma íntima con las reglas de estilo de código. El cumplimiento de la inmutabilidad local de las variables (`LocalVariableCouldBeFinal`) y la eliminación del código muerto de constructores redundantes (`UnnecessaryConstructor`) facilitan la **Analizabilidad** e **Inteligibilidad** del código fuente, reduciendo el esfuerzo cognitivo que requiere un nuevo programador para comprender la arquitectura del ajedrez. Asimismo, simplifica la **Capacidad de modificación** del sistema de cara a futuros desarrollos.

La característica de **Fiabilidad** se mapea con las advertencias de la categoría Error Prone. La identificación de asignaciones de objetos a nulo (`NullAssignment`) y comparaciones de objetos incorrectas (`CompareObjectsWithEquals`) ayuda a prevenir excepciones en tiempo de ejecución, lo que impacta de manera positiva en la **Madurez** y la **Tolerancia a fallos** de la aplicación del ajedrez.

La característica de **Eficiencia de desempeño** se asocia con las reglas de rendimiento. Aunque en el contexto actual del proyecto de ajedrez la creación de objetos en bucles tiene un impacto bajo, esta dimensión de calidad sería crítica si se implementase en el futuro un motor de inteligencia artificial de búsqueda en árbol de jugadas. En este caso, evitar la instanciación de objetos efímeros dentro de los algoritmos de poda asegurarían un correcto **Comportamiento temporal** y una óptima **Utilización de recursos**.

Cabe destacar la limitación inherente de las herramientas de análisis estático: PMD audita la calidad estructural interna del código fuente, pero es incapaz de verificar la corrección lógica y el comportamiento de la aplicación. Para asegurar de manera formal la adecuación funcional (por ejemplo, que la torre no salte piezas o que el caballo se mueva en L), es obligatorio diseñar y ejecutar pruebas unitarias dinámicas de caja blanca utilizando el framework de pruebas **JUnit 5**.

9. Conclusiones y resolución de hallazgos

La integración y configuración del plugin de PMD en el proyecto de ajedrez se ha completado satisfactoriamente, utilizando un ruleset personalizado y filtrando las reglas de complejidad sintáctica para enfocar los reportes en problemas de calidad estructural. Los reportes oficiales originales del programa (`target/pmd-original.xml` y `target/reports/pmd-original.html`) sirvieron inicialmente como base para identificar un total de 57 violaciones sintácticas y de diseño en el modelo y controlador.

Para garantizar la máxima calidad y madurez del código fuente antes de su entrega definitiva, se ha procedido a aplicar de manera íntegra todas las refactorizaciones y mejoras sugeridas en el análisis:

Resolución integral de las violaciones aplicadas en el código

1. **Inmutabilidad local aplicada:** Se modificaron las 38 variables locales no reasignadas reportadas por la regla `LocalVariableCouldBeFinal` a lo largo del modelo y los controladores, declarándolas con el modificador `final` para asegurar la inmutabilidad.
2. **Control de referencias nulas resuelto:** Se refactorizó la lógica interna de `CasillaSeleccionada` en la clase `ControladorJuego.java` para utilizar el tipo de datos genérico `Optional<Casilla>`, eliminando por completo el olor a código de las asignaciones de variables a nulo en el flujo del juego.
3. **Pruebas unitarias de adecuación funcional:** Se ha diseñado e implementado una suite completa de pruebas automatizadas utilizando el framework **JUnit 5** y configurado la última versión estable del plugin `maven-surefire-plugin`. Los tests validan el comportamiento dinámico del movimiento de las piezas (Alfil, Caballo, Peón, Reina, Rey y Torre) en el tablero, y se ejecutan con éxito obteniendo un rotundo 'BUILD SUCCESS' de manera consistente.

4. **Activación de la puerta de calidad estricta:** Tras comprobar que el código quedó completamente limpio de las alertas de estilo y de tipado de objetos de PMD (0 violaciones restantes de las 57 originales), se configuró de forma permanente la directiva `failOnViolation` a `true` en el archivo `pom.xml`. A partir de este momento, PMD actúa como una puerta de calidad infranqueable para blindar la build de Maven ante regresiones en futuras entregas.

Anexo A: Evidencias de salida

```
[INFO] Scanning for projects...
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ base
---
[INFO] Recompiling the module because of source changes.
[INFO] Compiling 21 source files to .../ajedrez/target/classes
[INFO] --- maven-pmd-plugin:3.26.0:check (default-cli) @ base ---
[INFO] PMD Version: 7.7.0
[WARNING] PMD Failure: com.ajedrez.controlador.ControladorJuego:181
    Rule:NullAssignment Priority:3 Assigning an Object to null is a code
    smell. Consider refactoring..
[WARNING] PMD Failure: com.ajedrez.modelo.piezas.Peon:83
    Rule:CompareObjectsWithEquals Priority:3 Use equals() to compare
    object references..
[INFO]
-----
[INFO] BUILD SUCCESS
[INFO]
-----
```

Listing 1: Salida reducida del proceso de compilacion y PMD

```
<?xml version="1.0" encoding="UTF-8"?>
<pmd xmlns="http://pmd.sourceforge.net/report/2.0.0" version="7.7.0">
  <file name=".../Peon.java">
    <violation beginline="83" endline="83"
      rule="CompareObjectsWithEquals" ruleset="Error Prone" priority="3">
      Use equals() to compare object references.
    </violation>
  </file>
</pmd>
```

Listing 2: Detalle de los hallazgos en target/pmd.xml

Anexo B: Materiales y bibliografía

- **PMD Source Code Analyzer - Documentación Oficial:** <https://pmd.github.io/>
- **Apache Maven PMD Plugin Integration Reference:** <https://maven.apache.org/plugins/maven-pmd-plugin/>

- **Portal Oficial ISO/IEC 25000 SQuaRE:** <https://iso25000.com/>
- **Apuntes de Auditoría e Certificación da Calidade de SI (ACCSI):** Temas 3 y 4, Curso 2025/26.